DATA 211
Fundamentals of Data Science

LECTURE 2: NUMPY & MATPLOTLIB

Why Python?

- Popular Choice
  - Widely used in data science due to simplicity and power.
- Powerful Libraries
  - Supports powerful libraries for data science.
- Great for Data Wrangling
  - Clean, reshape, and transform data with ease.
- Strong Community & Resources
  - Active forums, tutorials, and open-source tools.
- Integration & Automation
  - Easily integrates with databases, APIs, and dashboards.

Install Libraries

Individually:

- pip install seaborn

- pip install numpy

- pip install pandas

- pip install jupyter

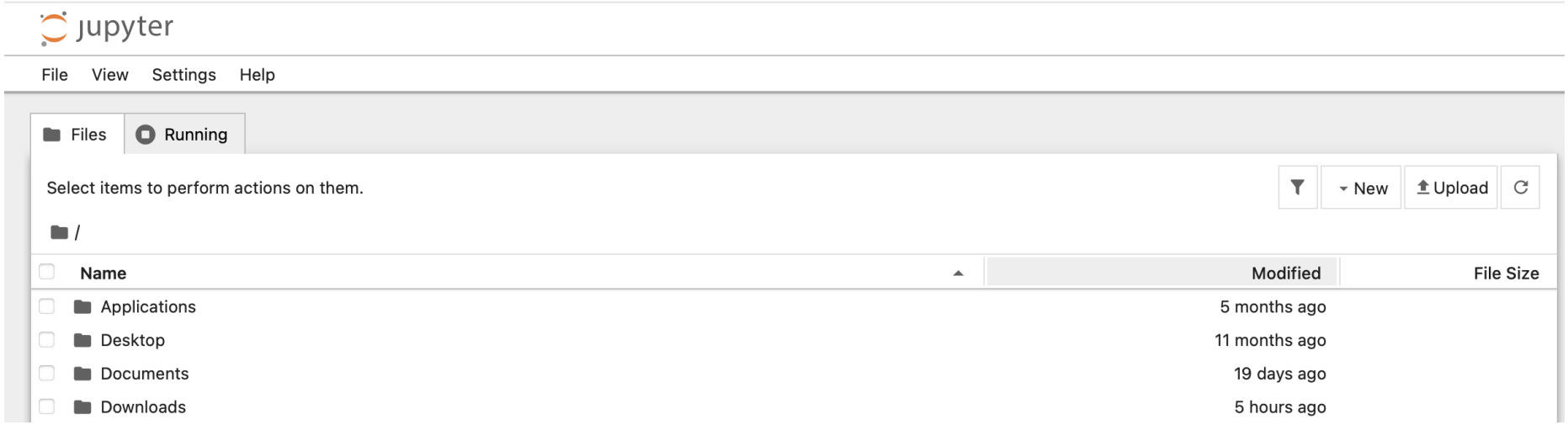
All together (common)

- **pip install numpy pandas matplotlib seaborn jupyter jupyterlab notebook**

- pip install mlxtend scipy scikit-learn

Running **Jupyter Notebook**

- **jupyter notebook**



1. Numpy

BASIC INTRODUCTION

Numpy

- Core Library for Numerical Computing
  - Efficient handling of arrays, matrices, and mathematical functions.
- Key Features
  - ndarray – multi-dimensional array object
  - Fast element-wise operations
  - Broadcasting for efficient computation
  - Tools for linear algebra, Fourier transform, and random number generation
- Why Use NumPy?
  - Much faster than native Python lists
  - Foundation for many other data science libraries (like Pandas, SciPy)

Creating a Basic NumPy Array

- The core object in NumPy is the *ndarray*, which stands for N-dimensional array.

```python
import numpy as np

# The main object is the homogeneous multidimensional array (ndarray)
arr = np.array([1, 2, 3])
print("Simple 1D array:", arr)
print("Type:", type(arr))
print("Data type:", arr.dtype)

arr = np.array([[1, 2], [3, 4]])
print("2D Array:\n", arr)
```

```
Simple 1D array: [1 2 3]
Type: <class 'numpy.ndarray'>
Data type: int64
2D Array:
 [[1 2]
 [3 4]]
```

Creating Special Arrays

- NumPy includes many utility functions to create arrays filled with specific values or patterns.

```
# Special arrays
zeros = np.zeros((2, 3))            # Array of zeros
ones = np.ones((2, 2))              # Array of ones
identity = np.eye(3)                # Identity matrix
range_arr = np.arange(0, 10, 2)     # Like range() but returns array
linspace = np.linspace(0, 1, 5)     # 5 numbers from 0 to 1 (inclusive)
random = np.random.random((2, 2))   # Random values between 0 and 1

print("Zeros array:\n", zeros)
print("Linear spaced array:", linspace)
```

```
Zeros array:
 [[0. 0. 0.]
 [0. 0. 0.]]
Linear spaced array: [0.   0.25 0.5  0.75 1.  ]
```

Inspecting Array Attributes

- Once you create a NumPy array, you can inspect various attributes that describe its structure and memory usage.

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

print("Array:\n", arr)
print("Dimensions:", arr.ndim)       # Number of dimensions
print("Shape:", arr.shape)           # Tuple of array dimensions
print("Size:", arr.size)             # Total number of elements
print("Data type:", arr.dtype)       # Data type of elements
print("Item size:", arr.itemsize)    # Size in bytes of each element
print("Data buffer:", arr.data)      # Python buffer object
```

```
Array:
 [[1 2 3]
 [4 5 6]]
Dimensions: 2
Shape: (2, 3)
Size: 6
Data type: int64
Item size: 8
Data buffer: <memory at 0x00000297DC2D3C60>
```

Indexing and Slicing Arrays

- NumPy allows powerful and intuitive ways to access individual elements or slices of data from arrays using standard Python indexing syntax.

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Indexing
print("Element at [0,1]:", arr[0, 1])  # First row, second column

# Slicing
print("First row:", arr[0, :])        # First row, all columns
print("Last column:", arr[:, -1])     # All rows, last column
print("Subarray:\n", arr[1:, 1:])     # Rows from index 1, columns from index 1

# Step slicing
print("Every other row:\n", arr[::2])  # Every other row
```

```
Element at [0,1]: 2
First row: [1 2 3]
Last column: [3 6 9]
Subarray:
 [[5 6]
 [8 9]]
Every other row:
 [[1 2 3]
 [7 8 9]]
```

Reshaping & Flattening Arrays

- Numpy provides you with methods to reshape and simplify array structures when preparing data for computation or visualization.

```
arr = np.arange(12)
print("Original array:", arr)

# Reshape (returns new view if possible)
reshaped = arr.reshape(3, 4)
print("Reshaped (3x4):\n", reshaped)

# Resize (modifies array in-place)
arr.resize(4, 3)
print("Resized (4x3):\n", arr)

# Flatten (always returns copy)
flat = arr.flatten()
print("Flattened:", flat)

# Ravel (returns view when possible)
raveled = arr.ravel()
print("Raveled:", raveled)
```

```
Original array: [ 0  1  2  3  4  5  6  7  8  9 10 11]

Reshaped (3x4):
 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]

Resized (4x3):
 [[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
Flattened: [ 0  1  2  3  4  5  6  7  8  9 10 11]

Raveled: [ 0  1  2  3  4  5  6  7  8  9 10 11]
```

Creating Arrays with Specific Types

- NumPy supports a wide range of data types and allows precise control over them. This is especially useful when optimizing memory or interfacing with low-level code.

```
# Creating arrays with specific types
int_arr = np.array([1, 2, 3], dtype=np.int32)
float_arr = np.array([1, 2, 3], dtype=np.float64)

print("Integer array:", int_arr.dtype)
print("Float array:", float_arr.dtype)

# Type conversion
converted = int_arr.astype(np.float64)
print("Converted to float:", converted.dtype)
```

```
Integer array: int32
Float array: float64
Converted to float: float64
```

Mathematical Operations

- NumPy supports both element-wise and matrix operations using intuitive syntax.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Element-wise operations
print("Addition:", a + b)
print("Multiplication:", a * b)
print("Division:", b / a)
print("Exponentiation:", a ** 2)

# Matrix multiplication
mat_a = np.array([[1, 2], [3, 4]])
mat_b = np.array([[5, 6], [7, 8]])
print("Matrix multiplication:\n", np.dot(mat_a, mat_b))
print("Alternative syntax:\n", mat_a @ mat_b)

# Universal functions (ufuncs)
print("Square root:", np.sqrt(a))
print("Exponential:", np.exp(a))
print("Trigonometric:", np.sin(a))
```

```
Addition: [5 7 9]
Multiplication: [ 4 10 18]
Division: [4.  2.5 2. ]
Exponentiation: [1 4 9]

Matrix multiplication:
 [[19 22]
 [43 50]]
Alternative syntax:
 [[19 22]
 [43 50]]

Square root: [1.         1.41421356 1.73205081]
Exponential: [ 2.71828183  7.3890561  20.08553692]
Trigonometric: [0.84147098 0.90929743 0.14112001]
```

Aggregation Functions

- NumPy provides fast aggregation functions that operate across entire arrays or along specified axes.

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

print("Sum of all elements:", np.sum(arr))
print("Sum along columns:", np.sum(arr, axis=0))
print("Sum along rows:", np.sum(arr, axis=1))

print("Mean:", np.mean(arr))
print("Standard deviation:", np.std(arr))
print("Minimum:", np.min(arr))
print("Maximum:", np.max(arr))

print("Index of max value:", np.argmax(arr))
print("Cumulative sum:", np.cumsum(arr))
```

```
Sum of all elements: 21
Sum along columns: [5 7 9]
Sum along rows: [ 6 15]
Mean: 3.5
Standard deviation: 1.707825127659933
Minimum: 1
Maximum: 6
Index of max value: 5
Cumulative sum: [ 1  3  6 10 15 21]
```

Broadcasting in NumPy

- Broadcasting allows NumPy to perform arithmetic operations on arrays of different shapes, without explicit looping.

```python
# Broadcasting allows operations between arrays of different shapes

# Scalar broadcasting
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Array + scalar:\n", arr + 10)

# Vector broadcasting
row = np.array([10, 20, 30])
print("Array + row vector:\n", arr + row)

# Column broadcasting
col = np.array([[10], [20]])
print("Array + column vector:\n", arr + col)
```

```
Array + scalar:
 [[11 12 13]
 [14 15 16]]
Array + row vector:
 [[11 22 33]
 [14 25 36]]
Array + column vector:
 [[11 12 13]
 [24 25 26]]
```

Boolean Indexing in NumPy

- Boolean indexing allows you to filter, select, or modify elements in an array based on logical conditions.

```python
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Boolean array
bool_arr = arr > 5
print("Boolean array:\n", bool_arr)

# Using boolean array to filter elements
print("Elements > 5:", arr[arr > 5])

# Multiple conditions
print("Elements between 3 and 7:", arr[(arr > 3) & (arr < 7)])

# Setting values with boolean indexing
arr[arr > 5] = 0
print("After setting >5 to 0:\n", arr)
```

```
Boolean array:
 [[False False False]
 [False False  True]
 [ True  True  True]]
Elements > 5: [6 7 8 9]
Elements between 3 and 7: [4 5 6]
After setting >5 to 0:
 [[1 2 3]
 [4 5 0]
 [0 0 0]]
```

Fancy Indexing in NumPy

- Fancy indexing lets you access array elements using integer arrays or boolean masks, offering more control than basic slicing.

```python
arr = np.array([10, 20, 30, 40, 50])

# Using integer arrays for indexing
indices = [1, 3, 4]
print("Selected elements:", arr[indices])

# Multi-dimensional example
arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
row_indices = [0, 1, 2]
col_indices = [1, 0, 2]
print("Selected elements:", arr2d[row_indices, col_indices])

# Using boolean masks
mask = np.array([1, 0, 1, 0, 1], dtype=bool)
print("Masked selection:", arr[mask])
```

```
Selected elements: [20 40 50]
Selected elements: [2 4 9]
Masked selection: [10 30 50]
```

Iterating Over Arrays

- NumPy provides multiple ways to iterate over arrays, offering control over element access, traversal order, and efficiency.

```python
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Default iteration is row-wise for 2D arrays
print("Row iteration:")
for row in arr:
    print(row)

# Element-wise iteration (flattened)
print("Element iteration:")
for element in arr.flat:
    print(element)

# Using np.nditer for more control
print("nditer (column-major order):")
for x in np.nditer(arr, order='F'):  # Fortran (column-major) order
    print(x)

# Enumerated iteration
print("Enumerated iteration:")
for index, value in np.ndenumerate(arr):
    print(index, value)
```

```
Row iteration:
[1 2 3]
[4 5 6]
Element iteration:
1
2
3
4
5
6
nditer (column-major order):
1
4
2
5
3
6
Enumerated iteration:
(0, 0) 1
(0, 1) 2
(0, 2) 3
(1, 0) 4
(1, 1) 5
(1, 2) 6
```

Views and Copies

- A view shares the same data buffer as the original array. A copy, on the other hand, is a completely separate object with its own data.

```
arr = np.array([1, 2, 3, 4, 5])

# View (shallow copy)
view = arr.view()
view[0] = 100
print("Original after view modification:", arr)  # Original changes

# Copy (deep copy)
copy = arr.copy()
copy[0] = 999
print("Original after copy modification:", arr)  # Original unchanged

# Slicing creates views
slice_view = arr[1:4]
slice_view[0] = 200
print("Original after slice modification:", arr)  # Original changes

# Check if array is a view
print("Is view:", slice_view.base is arr)
```

```
Original after view modification: [100   2   3   4   5]
Original after copy modification: [100   2   3   4   5]
Original after slice modification: [100 200   3   4   5]
Is view: True
```

Stacking and Splitting Arrays

- NumPy provides convenient functions to combine (stack) and divide (split) arrays along different axes.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Stacking arrays
vertical = np.vstack((a, b))  # Stack vertically (row-wise)
horizontal = np.hstack((a, b))  # Stack horizontally (column-wise)
depth = np.dstack((a, b))  # Stack along third axis (depth)
print("Vertical stack:\n", vertical)
print("Horizontal stack:", horizontal)
```

```
Vertical stack:
 [[1 2 3]
 [4 5 6]]
Horizontal stack: [1 2 3 4 5 6]
```

Stacking and Splitting Arrays

- NumPy provides convenient functions to combine (stack) and divide (split) arrays along different axes.

```
# Splitting arrays
arr = np.arange(9).reshape(3, 3)
split_h = np.hsplit(arr, 3)  # Split horizontally
split_v = np.vsplit(arr, 3)  # Split vertically
print("Horizontal split:", split_h)
print("Vertical split:", split_v)
```

```
Horizontal split: [array([[0],
       [3],
       [6]]), array([[1],
       [4],
       [7]]), array([[2],
       [5],
       [8]])]
Vertical split: [array([[0, 1, 2]]), array([[3, 4, 5]]), array([[6, 7, 8]])]
```

Linear Algebra with NumPy

- NumPy provides powerful linear algebra tools through its linalg module. These include operations such as matrix multiplication, determinant calculation, solving linear systems, and more.

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

# Matrix multiplication
print("Matrix multiplication:\n", np.matmul(a, b))

# Dot product
print("Dot product:", np.dot(a, b))

# Determinant
print("Determinant:", np.linalg.det(a))
```

```
Matrix multiplication:
 [[19 22]
 [43 50]]
Dot product: [[19 22]
 [43 50]]
Determinant: -2.0000000000000004
```

Linear Algebra with NumPy

- NumPy provides powerful linear algebra tools through its linalg module. These include operations such as matrix multiplication, determinant calculation, solving linear systems, and more.

```
# Inverse
print("Inverse:\n", np.linalg.inv(a))

# Eigenvalues and eigenvectors
eigvals, eigvecs = np.linalg.eig(a)
print("Eigenvalues:", eigvals)
print("Eigenvectors:\n", eigvecs)

# Solving linear equations
# 1x + 2y = 5
# 3x + 4y = 6
solution = np.linalg.solve(a, np.array([5, 6]))
print("Solution to linear equations:", solution)
```

```
Inverse:
 [[-2.   1. ]
 [ 1.5 -0.5]]
Eigenvalues: [-0.37228132  5.37228132]
Eigenvectors:
 [[-0.82456484 -0.41597356]
 [ 0.56576746 -0.90937671]]
Solution to linear equations: [-4.   4.5]
```

Random Number Generation

- NumPy's np.random module provides functions for generating random numbers, including uniform distributions, normal distributions, and shuffling.

```
# Random numbers
print("Random float [0,1):", np.random.rand())
print("Random floats in array:", np.random.rand(3, 2))

# Random integers
print("Random integers:", np.random.randint(0, 10, size=5))

# Random choice
print("Random choice:", np.random.choice(['a', 'b', 'c'], size=3))
```

```
Random float [0,1): 0.9699098521619943
Random floats in array: [[0.83244264 0.21233911]
 [0.18182497 0.18340451]
 [0.30424224 0.52475643]]
Random integers: [8 0 9 2 6]
Random choice: ['a' 'c' 'a']
```

Random Number Generation

- NumPy's np.random module provides functions for generating random numbers, including uniform distributions, normal distributions, and shuffling.

```
# Shuffling
arr = np.arange(10)
np.random.shuffle(arr)
print("Shuffled array:", arr)

# Normal distribution
print("Normal distribution:", np.random.normal(0, 1, 5))  # mean=0, std=1

# Setting random seed for reproducibility
np.random.seed(42)
print("With seed:", np.random.rand(3))
```

```
Shuffled array: [7 9 5 8 3 1 0 4 6 2]
Normal distribution: [-0.1402185  -0.0331934  -0.74907652 -0.77838201  0.94884286]
With seed: [0.37454012 0.95071431 0.73199394]
```

Saving and Loading Data

- NumPy provides convenient functions for saving and loading arrays from both text and binary formats.

```
# Saving and loading text files
arr = np.array([[1, 2, 3], [4, 5, 6]])
np.savetxt('array.txt', arr, delimiter=',')
loaded = np.loadtxt('array.txt', delimiter=',')
print("Loaded from text file:\n", loaded)

# Saving and loading binary files (more efficient)
np.save('array.npy', arr)
loaded_binary = np.load('array.npy')
print("Loaded from binary file:\n", loaded_binary)

# Saving multiple arrays
arr2 = np.array([7, 8, 9])
np.savez('arrays.npz', a=arr, b=arr2)
loaded_zip = np.load('arrays.npz')
print("Arrays from zip file:", loaded_zip['a'], loaded_zip['b'])
```

```
Loaded from text file:
 [[1. 2. 3.]
 [4. 5. 6.]]
Loaded from binary file:
 [[1 2 3]
 [4 5 6]]
Arrays from zip file: [[1 2 3]
 [4 5 6]] [7 8 9]
```

2. Matplotlib

BASIC INTRODUCTION

Introduction to Matplotlib

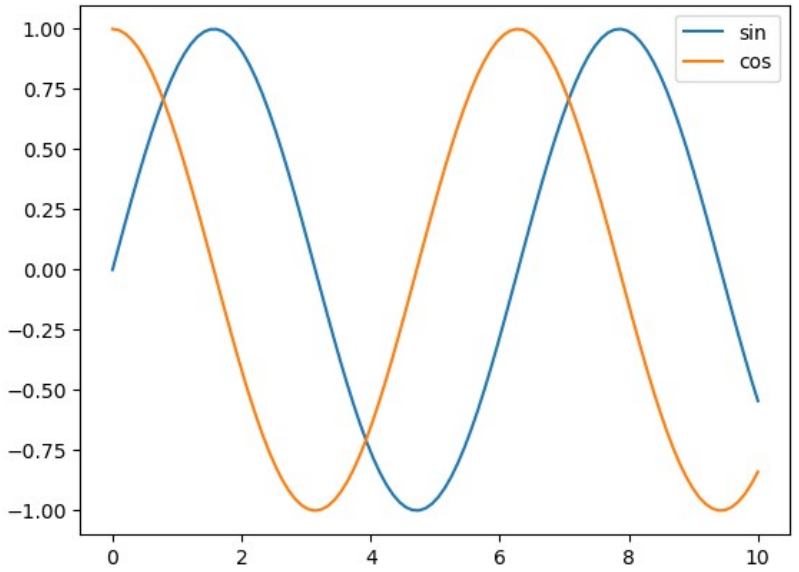
- Matplotlib is a powerful and widely used Python library for creating static, animated, and interactive visualizations.
- It provides an object-oriented API for embedding plots into applications and supports a variety of plot types such as line plots, bar charts, histograms, scatter plots, and more.

```
import matplotlib.pyplot as plt
import matplotlib.style as style
import seaborn as sns
import pandas as pd
import numpy as np
```

Line Plot using Matplotlib

- You can create a simple line plot with x and y coordinates using plt.plot().

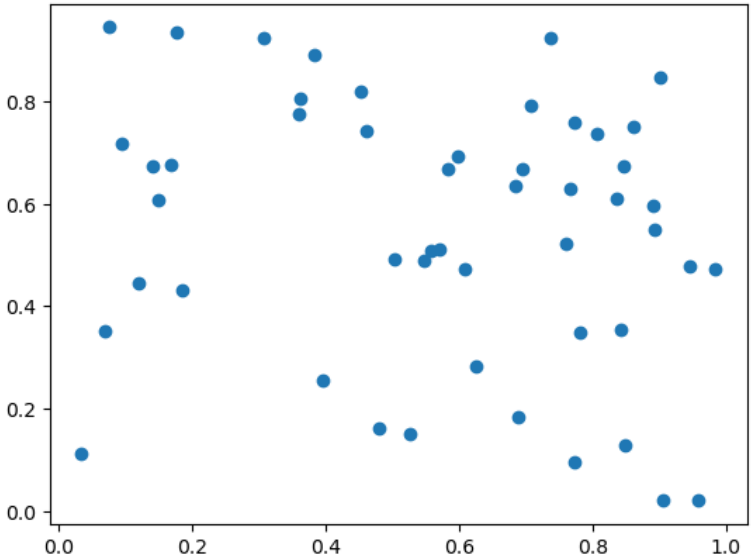
```
# Multiple line plots
plt.plot(x, np.sin(x), label='sin')
plt.plot(x, np.cos(x), label='cos')
plt.legend()
plt.show()
```



Scatter Plot

- You can generate a scatter plot to visualize point distributions with plt.scatter().

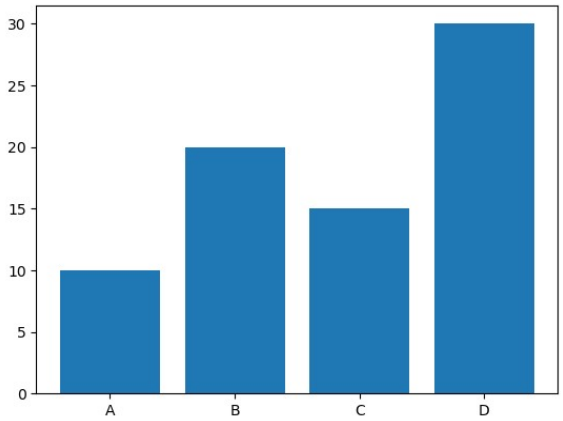
```
# Scatter plot
x = np.random.rand(50)
y = np.random.rand(50)
plt.scatter(x, y)
plt.show()
```



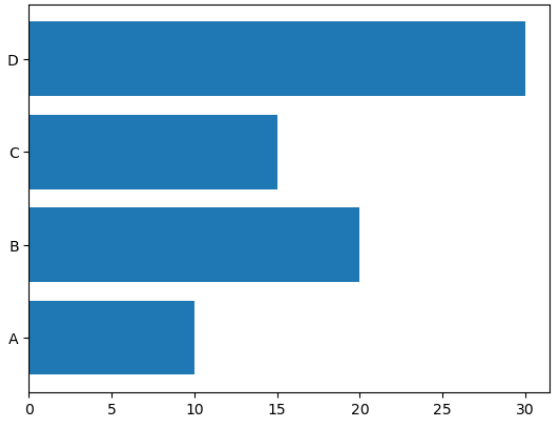
Bar Chart

- You can build a vertical bar chart comparing categorical data with plt.bar().

```
# Bar chart
categories = ['A', 'B', 'C', 'D']
values = [10, 20, 15, 30]
plt.bar(categories, values)
plt.show()
```



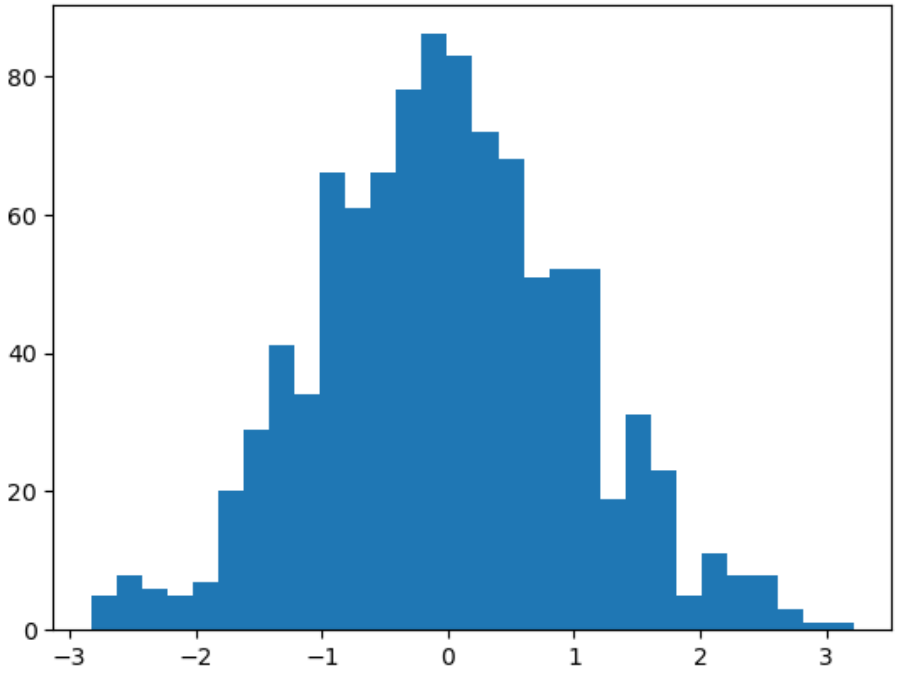

```
# Horizontal bar chart
plt.barh(categories, values)
plt.show()
```



Histogram

- You can display data distribution using bins with plt.hist().

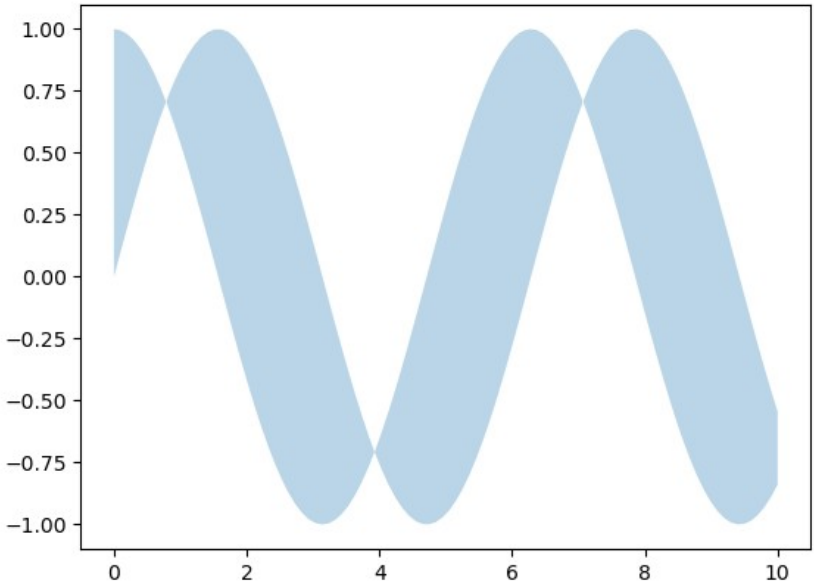
```
# Histogram
data = np.random.randn(1000)
plt.hist(data, bins=30)
plt.show()
```



Area Plot

- You can create stacked or unstacked area plots to visualize cumulative trends and part-to-whole relationships using plt.fill_between().

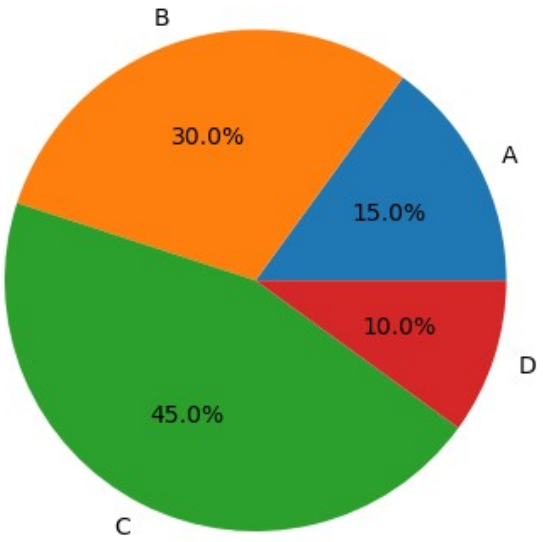
```
# Area plot
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
plt.fill_between(x, y1, y2, alpha=0.3)
plt.show()
```



Pie Chart

- You can create a pie chart to show proportional data with plt.pie().

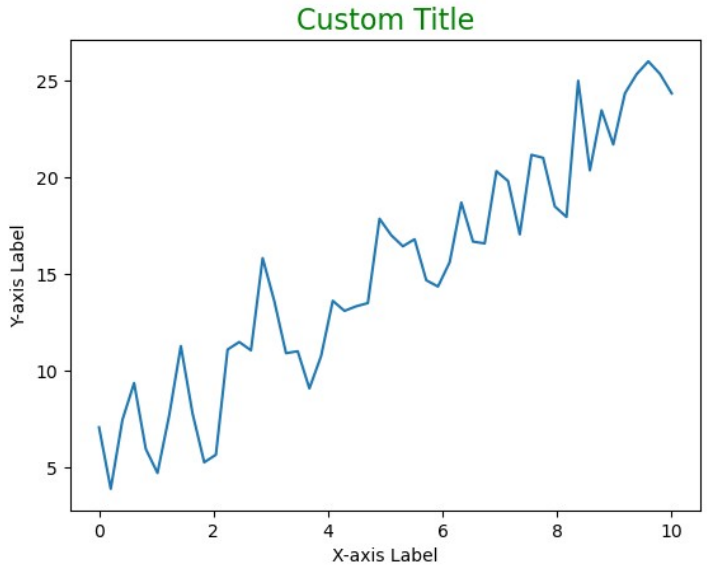
```
# Pie chart
sizes = [15, 30, 45, 10]
labels = ['A', 'B', 'C', 'D']
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.show()
```



Titles and Labels

- You can add the titles and labels to your graphs for more readability.

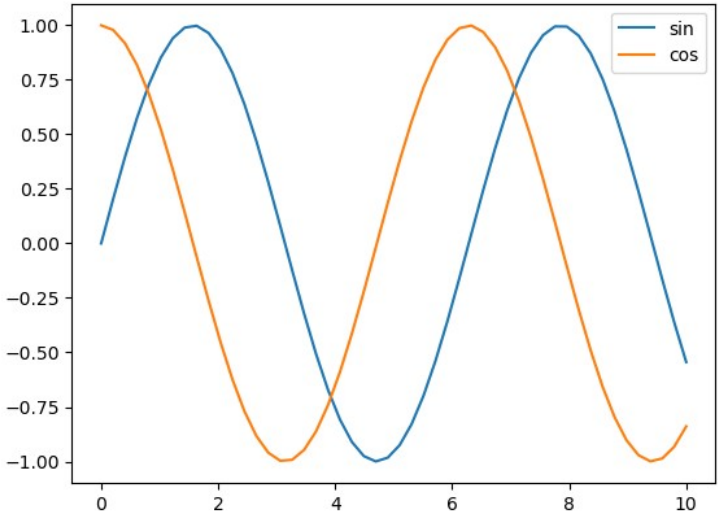
```
x = np.linspace(0, 10, 50)  # 50 values from 0 to 10
y = 2 * x + 5 + np.random.normal(0, 2, size=x.shape)  # y = 2x + 5 + noise
plt.plot(x, y)
plt.title('Custom Title', fontsize=16, color='green')
plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.show()
```



Legend

- A legend can be added to a graph using plt.legend() function.

```
# Legend
x = np.linspace(0, 10, 50)  # 50 values from 0 to 10
y = 2 * x + 5 + np.random.normal(0, 2, size=x.shape)  # y = 2x + 5 + noise
plt.plot(x, np.sin(x), label='sin')
plt.plot(x, np.cos(x), label='cos')
plt.legend(loc='upper right', fontsize=10)
plt.show()
```



Customizing Grid, Ticks, & Labels

- You can add customizations to a graph to make it presentation-ready with readable ticks, gridlines, and labels.

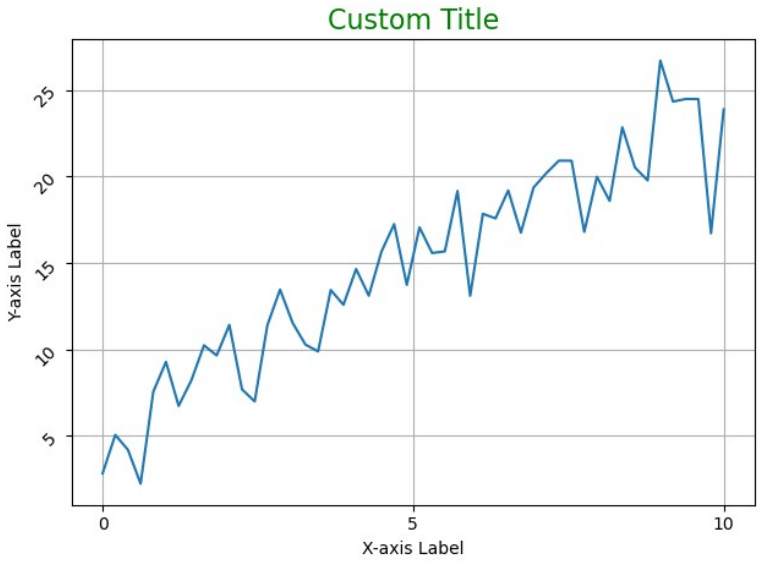
```
x = np.linspace(0, 10, 50)
y = 2 * x + 5 + np.random.normal(0, 2, size=x.shape)

fig, ax = plt.subplots()
ax.plot(x, y)

# Add grid and custom ticks
ax.grid(True)
ax.set_xticks([0, 5, 10])
ax.set_xticklabels(['Start', 'Middle', 'End'])
ax.tick_params(axis='y', labelrotation=45)

# Ensure ScalarFormatter is used before disabling scientific notation
ax.xaxis.set_major_formatter(ScalarFormatter())
ax.yaxis.set_major_formatter(ScalarFormatter())
ax.ticklabel_format(axis='both', style='plain')

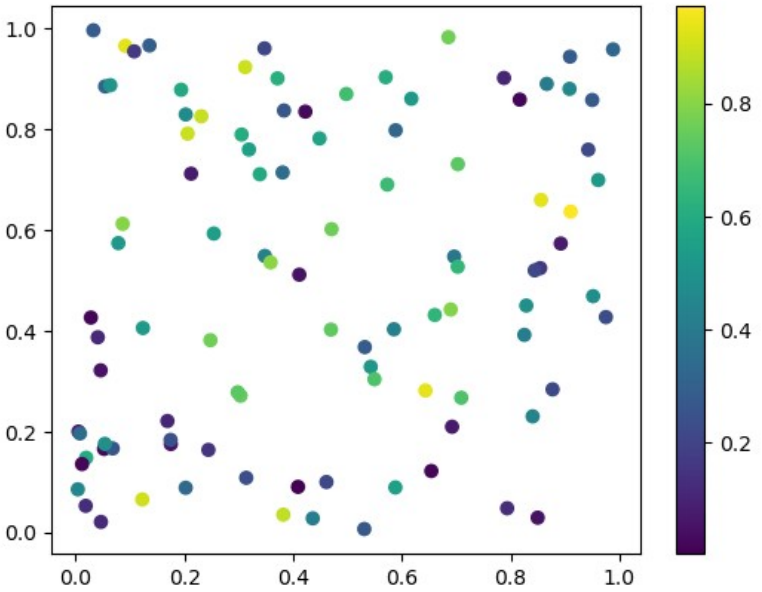
plt.title('Custom Title', fontsize=16, color='green')
plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.tight_layout()
plt.show()
```



Colorbar in a Scatter Plot

- You can use add a colorbar to the plot, providing a reference scale that maps color intensity to numeric values.

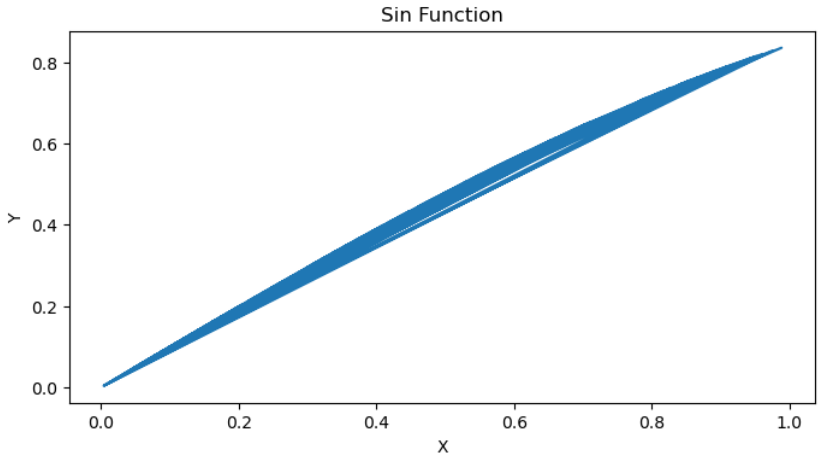
```
# Colorbar
x = np.random.rand(100)
y = np.random.rand(100)
colors = np.random.rand(100)
plt.scatter(x, y, c=colors, cmap='viridis')
plt.colorbar()
plt.show()
```



Plotting Using the OO Approach

- You can use Object-Oriented (OO) method for creating plots with Matplotlib, which offers more control and flexibility compared to the state-based (procedural) approach.

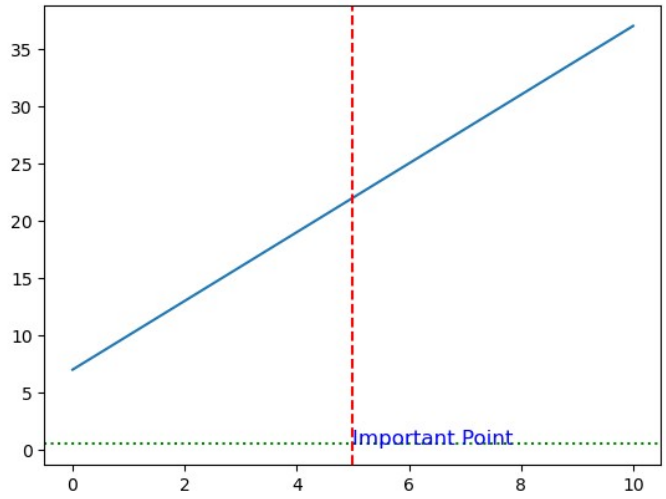
```
# Object-oriented approach
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(x, np.sin(x))
ax.set_title('Sin Function')
ax.set_xlabel('X')
ax.set_ylabel('Y')
plt.show()
```



Adding Text and Reference Lines

- You can annotate a plot using text and reference lines to highlight important values or thresholds.

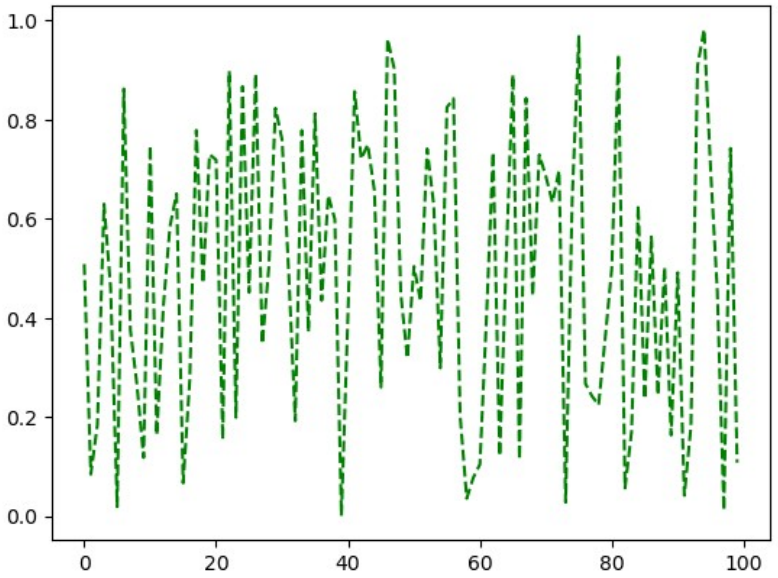
```
# Text and lines
fig, ax = plt.subplots()
ax.plot(x, y)
ax.text(5, 0.5, 'Important Point', fontsize=12, color='blue')
ax.axvline(x=5, color='red', linestyle='--')
ax.axhline(y=0.5, color='green', linestyle=':')
plt.show()
```



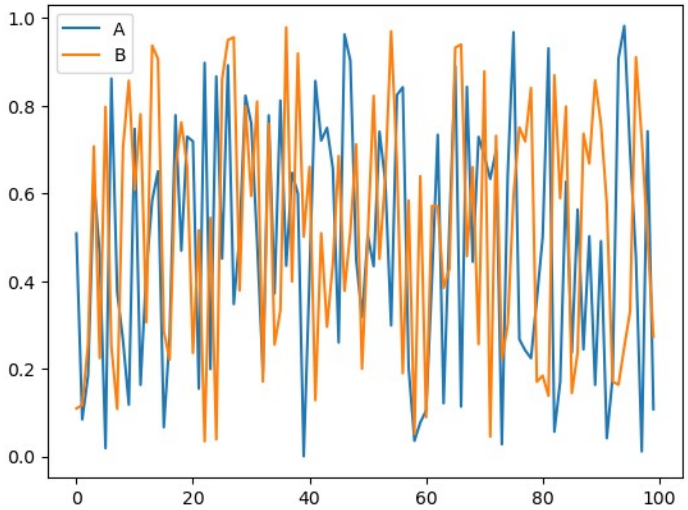
Line Plot from a Pandas Series

- The use of pandas' integrated plotting makes it quick and convenient to visualize Series data without explicitly using matplotlib.pyplot.plot()

```
# Line plot from Series
df['A'].plot.line(color='green', linestyle='--')
plt.show()
```



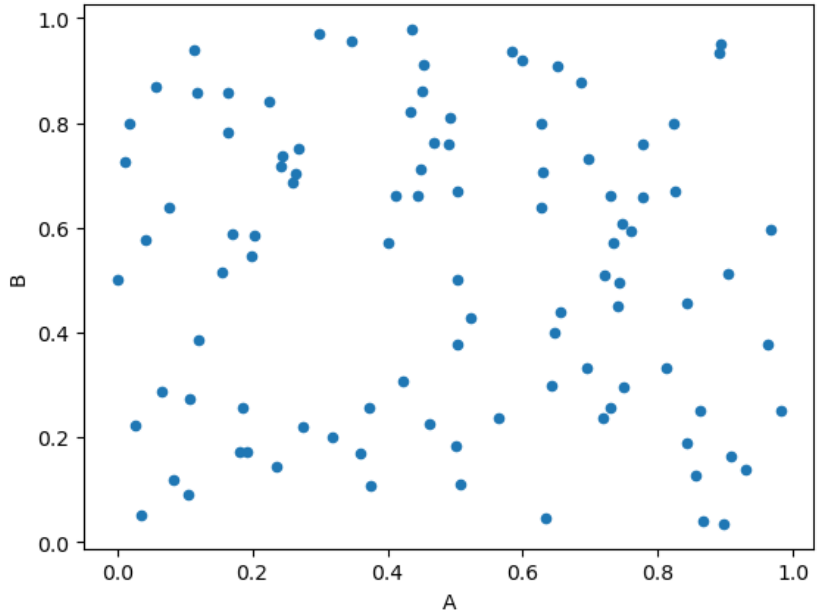

```
# Line plot from DataFrame
df[['A', 'B']].plot.line()
plt.show()
```



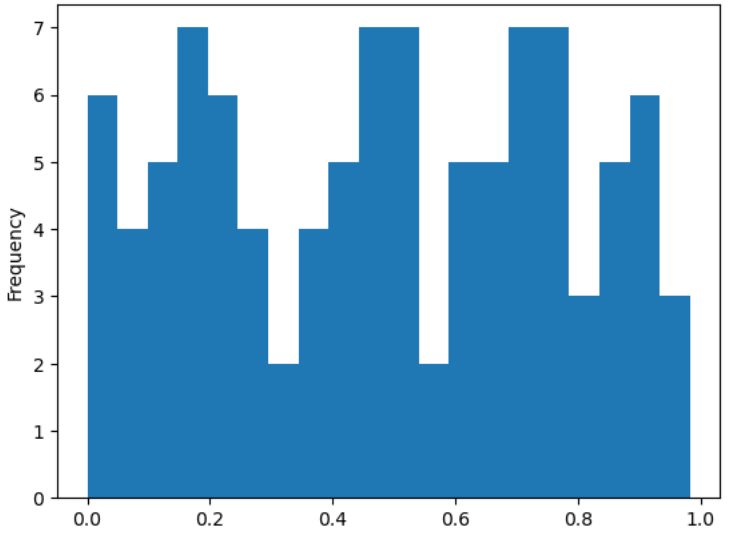
Scatter Plot & Histogram from a Pandas DataFrame

- df.plot.scatter() function can be used to draw a scatter plot from a dataframe.

```
# Scatter plot from DataFrame
df.plot.scatter(x='A', y='B')
plt.show()
```



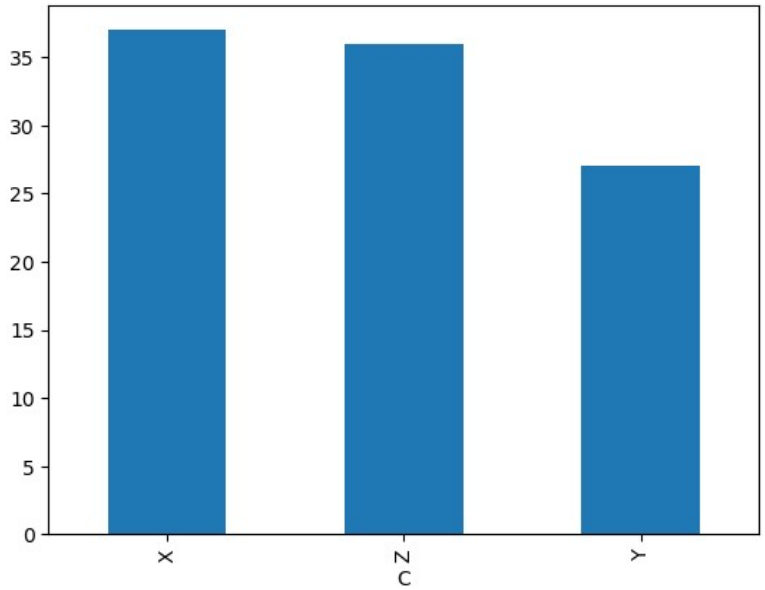

```
# Histogram from Series
df['A'].plot.hist(bins=20)
plt.show()
```



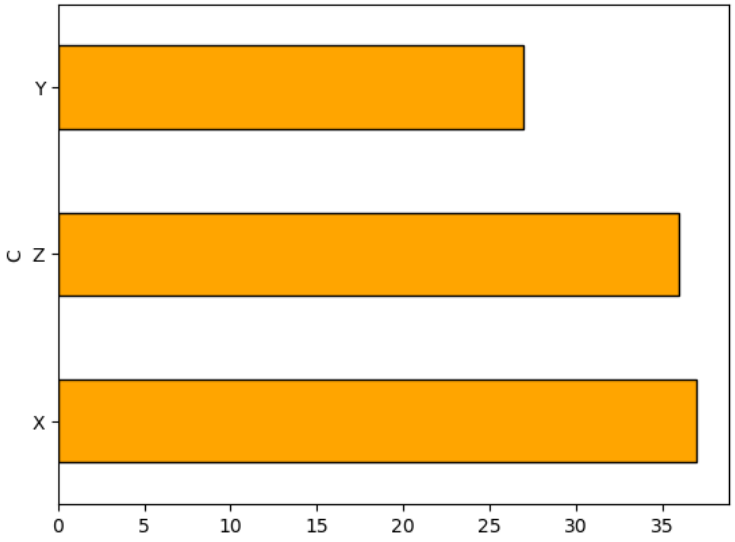
Bar Chart from a Pandas Series

- .bar() and .barh() functions can be used to draw the vertical and horizontal bar charts from a Series.

```
# Bar chart from Series
df['C'].value_counts().plot.bar()
plt.show()
```



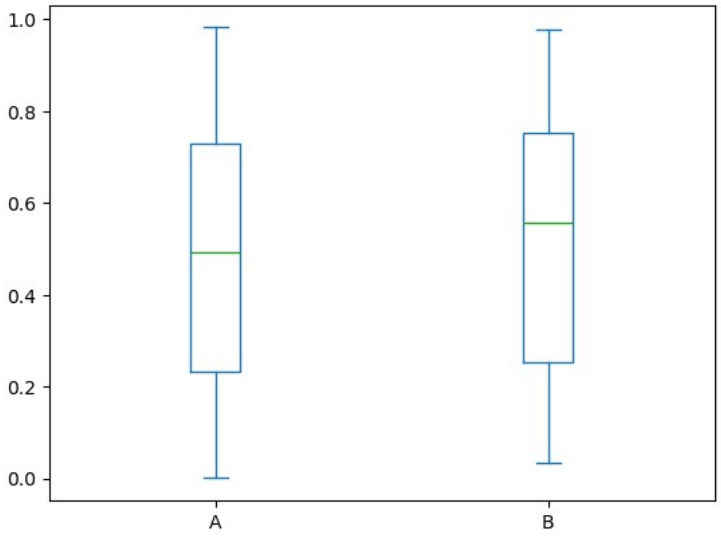

```
# Horizontal bar chart
df['C'].value_counts().plot.barh(
                                    color='orange',
                                   edgecolor='black')
plt.show()
```



Box Plot from DataFrame Columns

- Box plots are ideal for visualizing the distribution, spread, and presence of outliers in numerical data.

```
# Box plot
df[['A', 'B']].plot.box()
plt.show()
```



Thank You